Studio Nx

JS 33 — initSearch()

2026-10-08

JS 33 — `initSearch()` (+ `refreshList`)

항목	내용
파일	`studionx/frontend/static/js/pages/myproject.js`
작업	P1-14 내 프로젝트 화면 — 검색 · 정렬 · PRD 01 FR-123
작성	Claude (AI) · 2026-10-08 · 사용자 검토
이전 · 다음	32 `renderPager()` · 34 `initRowMenus()`

1. 하는 일

목록 위 검색 칸 · 정렬 칸이 바뀌면 **1쪽부터 목록을 다시 받아 그린다**.

사용자가 한 일	언제 요청	결과 (주소)
검색 칸에 `alp` 를 침	치기를 **멈추고 0.3초 뒤 한 번**	`alpine, Alpha Song` (`?query=alp`)
공백을 하나 더 침	요청 안 함 (검색어가 같음)	그대로
정렬을 Name 으로	**바로**	`Alpha Song, alpine` (`?query=alp&sort=title`)
검색 칸을 비움	0.3초 뒤	전체 4개 (`?sort=title`)
`gam` 을 치고 바로 Enter	**바로 · 한 번만** (기다리던 것은 취소)	`Gamma Loop`

함께 만든 것

- `refreshList(options)` — "받기(30) → 목록(31) → 페이지 줄(32)" 세 줄을 한 함수로. 32 페이지 넘김도 이것을 쓰도록 바꿨고, 35 · 37 도 쓴다.
- **30 `loadProjects` 에 두 줄 추가 — 늦게 온 옛 응답 버리기.** 아래 §5.

2. 언제 쓰나

호출하는 곳	화면
페이지 시작 (`myproject.js` 맨 아래, 이미 있음)	`initSearch();` — 검색 칸 · 정렬 칸에 듣기를 한 번 걸어 둠

디바운스(debounce): 글자를 칠 때마다 요청하지 않고, **입력이 멈춘 뒤** 정해진 시간이 지나면 한 번만 하는 방법. 칠 때마다 이전 타이머를 지우고(`clearTimeout`) 새로 건다(`setTimeout`) — 마지막 타이머만 살아남는다. `alp` 를 0.06초 간격으로 치면 요청은 3번이 아니라 1번.

3. 코드

import (`myproject.js` 맨 위 — 30 과 같음, 이번에 추가한 것 없음)

```
import { initAuthState } from '../core/auth-ui.js';
import { requireLogin, logout, loginUrl } from '../core/auth.js';
import { api } from '../core/api.js';
import { initModalFallback } from '../core/modal.js';
import { canOpenStudio, showDesktopOnly, initDesktopOnlyLinks, openStudio } from '../core/studio.js';
```

```
import { includePartials } from '../core/include.js';
import { initFormAlerts } from '../core/dialog.js';
```

이 함수는 import 를 쓰지 않는다 — 같은 파일의 `current` · `loadProjects` · `renderProjects` · `renderPager` 를 쓴다.

상수 (이번에 추가)

```
/** 검색 칸 입력을 멈춘 뒤 기다리는 시간 (ms) */
const SEARCH_DELAY = 300;

/** loadProjects 요청 번호 — 늦게 도착한 옛 응답을 버리려고 */
let loadTicket = 0;
```

함수

```
export function initSearch() {
  const form = document.getElementById('project-search');
  const input = document.getElementById('project-query');
  const sort = document.getElementById('project-sort');
  if (!form || !input || !sort) return;
  let timer = null;

  const search = function () {
    clearTimeout(timer);
    refreshList({ query: input.value, sort: sort.value, page: 1 });
  };

  input.addEventListener('input', function () {
    clearTimeout(timer);
    if (input.value.trim() === current.query) return;   // 공백만 더 쳤거나 원래대로 되돌림
    timer = setTimeout(search, SEARCH_DELAY);
  });
  sort.addEventListener('change', search);
  form.addEventListener('submit', function (event) {
    event.preventDefault();   // 페이지를 새로 열지 않음
    search();
  });
}

/** 목록 다시 받아 그리기 — loadProjects → renderProjects → renderPager (실패 · 옛 응답이면 아무것도 안 함) */
async function refreshList(options) {
  const data = await loadProjects(options);
  if (!data) return null;
  renderProjects(data);
  renderPager(data.total, data.page, data.size);
  return data;
}
```

30 `loadProjects` 에 추가한 부분

```
  const ticket = ++loadTicket;
  let data;
  try {
```

```
    data = await api.get('/projects?' + params.toString());
  } catch (err) {
    if (ticket !== loadTicket) return null;   // 더 새 요청이 있으면 이 실패는 무시
    …
  }
  if (ticket !== loadTicket) return null;   // 그사이 더 새 요청을 보냄 → 이 응답은 버림 (33)
```

최신 문법으로 쓰면 (공부용 — 바뀌는 부분만)

```
input.addEventListener('input', () => { … });
form.addEventListener('submit', (event) => { event.preventDefault(); search(); });
```

최신 문법	실제 코드	뜻
`() => { … }`	`function () { … }`	함수

문법 풀이

import 문법	뜻
(이 함수는 import 없음)	같은 파일의 함수를 부름

식	뜻	예
`setTimeout(search, SEARCH_DELAY)`	0.3초 뒤 `search` 를 한 번 부름. 번호(타이머)를 돌려줌	
`clearTimeout(timer)`	아직 안 불린 타이머 취소 (이미 지났거나 null 이면 아무 일 없음)	다음 글자를 치면 앞 타이머 취소
`input.value.trim() === current.query`	지금 목록의 검색어와 같으면 요청할 필요 없음	`'alp '` → `'alp'` 같음
`'input'` · `'change'` · `'submit'` 이벤트	글자가 바뀔 때마다 / select 값을 고른 뒤 / Enter (폼 보내기)	
`event.preventDefault()`	폼 기본 동작(주소에 `?query=` 를 붙여 페이지 새로 열기) 막기	
`++loadTicket`	1 늘린 **뒤의** 값 — 요청마다 새 번호	1, 2, 3 …
`ticket !== loadTicket`	내가 보낸 뒤에 다른 요청이 또 나갔는지	내 번호 1, 지금 2 → 옛 것

4. 흐름

- **글자 입력:** 타이머 취소 → 검색어가 지금과 같으면 끝 → 0.3초 타이머.
- **정렬 변경 · Enter:** 타이머 취소 → 바로 `refreshList({ query, sort, page: 1 })`.
- **`refreshList`:** 받기 → 실패 · 옛 응답이면 `null` 로 끝 → 목록 · 페이지 줄 그리기.

5. 왜 이렇게

선택	이유
0.3초 디바운스	템플릿 · HTML 주석의 값. 칠 때마다 요청하면 서버 부담 · 목록이 깜빡임
정렬 · Enter 는 바로	한 번 고르는 동작이라 기다릴 이유가 없음. Enter 는 "지금 찾아" 라는 뜻

선택	이유
검색어가 같으면 건너뜀	공백만 더 쳤거나 지웠다 다시 친 경우 같은 결과를 또 받지 않음
검색 · 정렬은 항상 1쪽부터	3쪽을 보다 검색하면 결과가 1쪽뿐일 수 있음
옛 응답 버리기 (30 에 추가)	`beta` 요청이 늦게 도착하면, 그 뒤에 보낸 `alpine` 결과를 덮어써 화면 · 주소가 `beta` 로 돌아간다. 요청마다 번호를 붙이고 마지막 번호의 응답만 쓴다. **시험에서 이 두 줄을 잠시 빼고 돌리면 실제로 `Beta Beat` 가 남는 것을 확인**하고 되돌렸다
`refreshList` 로 묶음	"받기 → 그리기 → 페이지 줄" 이 32 · 33 · 35 · 37 에서 반복됨
`type="search"` 칸의 × 버튼	누르면 `input` 이벤트가 나서 비운 것과 같게 동작 (따로 처리 없음)

6. 시험 (실제 myProject.html · 실제 서버 · 6개 + 앞 시험 다시)

프로젝트 4개(`Alpha Song`, `Beta Beat`, `Gamma Loop`, `alpine`)를 만들고, 글자 넣기 · 이벤트를 페이지 안에서 일으켰다. `window.fetch` 를 감싸 목록 요청 수를 셌다.

#	한 일	결과
01	`alp` 를 0.06초 간격으로	치는 동안 요청 0 → 멈춘 뒤 **1번** · `alpine, Alpha Song` · `?query=alp`
02	공백 하나 더	요청 0
03	정렬 Name	바로 1번 · `Alpha Song, alpine` · `?query=alp&sort=title`
04	검색 칸 비움	1번 · 4개 · `?sort=title`
05	`gam` + 바로 Enter	0.15초 안에 `Gamma Loop` · 끝까지 요청 **1번** (타이머 취소됨)
06	`beta` 요청을 1.2초 늦게 도착하게 → 그사이 `alpine` + Enter	화면 · 주소 `alpine` 그대로 (옛 응답 버림). 가드를 빼면 `Beta Beat` 로 덮임

앞 시험 다시 (30 · 32 를 고쳤으므로): 30 의 14개, 32 의 10개 모두 그대로 통과 — 페이지 넘김 클릭은 여전히 요청 1번 · `?page=2`.